HANZO CONSENSUS AI

Consensus Mechanisms for Distributed AI Training and Inference

Zach Kelling

Hanzo AI

March 2026

Abstract

We present the consensus mechanisms underlying Hanzo ACI (AI Chain Infrastructure), a decentralized compute network where GPU operators serve AI models and earn HANZO tokens for verified work. Unlike blockchains that achieve consensus on transaction ordering, ACI must achieve consensus on computational correctness: did a GPU node actually run the claimed model on the claimed input and produce the claimed output? We introduce three novel consensus components: (i) Proof of AI Work (PoAI), a probabilistic verification protocol where verifier nodes re-execute a random subset of inference requests to detect dishonest workers, (ii) GPU Cluster Consensus, a protocol for coordinating distributed training and tensor-parallel inference across untrusted GPU nodes with Byzantine fault tolerance, and (iii) Inference Attestation, an on-chain commitment scheme that binds inference outputs to specific model versions, input hashes, and GPU node identities. We prove that PoAI detects cheating with probability $1-(1-p)^k$ where p is the sampling rate and k is the number of verification rounds, and that the economic cost of cheating exceeds the expected profit under the HANZO staking and slashing mechanism [2]. The system is deployed on Hanzo ACI with 240 GPU nodes, processing 47M inference requests per month with a measured cheating detection rate of 99.97% for a 5% sampling rate.

1 Introduction

Decentralized AI compute markets face a verification problem absent from traditional blockchains. In Bitcoin, verifying a transaction requires checking a digital signature—a deterministic operation costing microseconds. In a decentralized AI network, verifying an inference result requires re-executing the model forward pass—a stochastic operation costing milliseconds to seconds on expensive GPU hardware.

Full verification (re-executing every request) doubles compute cost and negates the economic benefits of decentralization. No verification allows rational operators to return random outputs, collect fees, and save GPU cycles. The challenge is designing a verification protocol that:

1. Makes cheating economically irrational under the HANZO staking model.
2. Imposes minimal overhead (<10% of total compute).
3. Handles the non-determinism of GPU floating-point arithmetic.
4. Scales to millions of inference requests per day.

This paper describes the three consensus mechanisms deployed on Hanzo ACI [1] that address these requirements.

2 System Model

Definition 1 (ACI Network)**.** *The ACI network consists of three node types:*

- ***Workers*** $\mathcal{W} = \{w_1, \ldots, w_W\}$*: GPU nodes that execute inference and training requests.*
- ***Verifiers*** $\mathcal{V} = \{v_1, \ldots, v_V\}$*: GPU nodes that re-execute a subset of requests for verification.*
- ***Routers*** $\mathcal{R} = \{r_1, \ldots, r_R\}$*: Nodes that route requests to workers and coordinate verification.*

All nodes stake HANZO tokens proportional to their declared capacity [2].

Assumption 1 (Byzantine Fault Model)**.** *At most $f < n/3$ of the verifier nodes in any verification committee are Byzantine (may deviate arbitrarily from the protocol). Workers may be rational (profit-maximizing) rather than Byzantine—they cheat only if cheating is profitable.*

Definition 2 (Inference Record)**.** *An inference record $\mathcal{I} = (m, H(\mathbf{x}), H(\mathbf{y}), w, \sigma_w, t)$ consists of model identifier m, input hash $H(\mathbf{x})$, output hash $H(\mathbf{y})$, worker identity w, worker signature σ_w, and timestamp t.*

3 Proof of AI Work

Proof of AI Work (PoAI) is a probabilistic verification protocol. Rather than re-executing every inference request, verifiers sample a random subset and check for consistency.

3.1 Verification Protocol

Algorithm 1 Proof of AI Work Verification

Require: Inference records $\{\mathcal{I}_1, \ldots, \mathcal{I}_N\}$ from epoch e, sampling rate p
Ensure: Verification result for each sampled record

```
S ← sample each I_i independently with probability p
for each I_i ∈ S in parallel do
    Retrieve input x_i from request log
    Re-execute: y'_i ← Infer(m_i, x_i, θ_i^det)
    Compare: δ_i = d(y_i, y'_i)
    if δ_i > ε then
        Flag worker w_i for potential cheating
    end if
end for
Submit verification results to on-chain consensus
```

3.2 Handling Non-Determinism

GPU floating-point arithmetic is non-deterministic across different hardware, driver versions, and even consecutive runs on the same GPU. Two correct executions of the same model on the same input may produce slightly different outputs.

Definition 3 (Verification Distance)**.** *The verification distance $d(\mathbf{y}, \mathbf{y}')$ between two output sequences uses a composite metric:*

$$d(\mathbf{y}, \mathbf{y}') = \alpha \cdot d_{token}(\mathbf{y}, \mathbf{y}') + (1 - \alpha) \cdot d_{logit}(\mathbf{y}, \mathbf{y}'), \tag{1}$$

where d_{token} is normalized edit distance on generated tokens and d_{logit} is KL divergence on output logit distributions.

To ensure deterministic verification, we fix the random seed and use temperature $\tau = 0$ (greedy decoding) for verification re-execution:

$$\theta^{\text{det}} = (\tau = 0, \text{seed} = H(\mathcal{I}_i)), \tag{2}$$

where the seed is derived from the inference record hash, making verification reproducible.

Theorem 1 (Detection Probability)**.** *For a worker that cheats on fraction χ of requests, the probability of detecting the cheating in a single epoch with sampling rate p and N total requests is:*

$$P_{detect} = 1 - (1-p)^{\chi N}. \tag{3}$$

For $p = 0.05$ (5% sampling), $\chi = 0.10$ (cheating on 10% of requests), and $N = 10000$ (requests per epoch): $P_{detect} > 1 - 10^{-22}$.

3.3 Economic Security

Definition 4 (Cheating Profitability)**.** *A rational worker cheats if the expected profit from cheating exceeds the expected loss from detection:*

$$\chi \cdot R_{save} > P_{detect} \cdot S_{slash}, \tag{4}$$

where R_{save} is compute cost saved per cheated request and S_{slash} is the slashing penalty.

Theorem 2 (Economic Security Bound)**.** *Under the HANZO staking parameters [2] (25% slashing for detected cheating, minimum stake proportional to GPU capacity), cheating is unprofitable for any cheating rate $\chi > 0$ when the sampling rate satisfies:*

$$p > \frac{\ln(R_{save}/S_{slash})}{\chi \cdot N}. \tag{5}$$

With production parameters ($R_{save}/S_{slash} \approx 0.001$, $N = 10000$): $p > 0.007$ suffices. We use $p = 0.05$ for a safety margin.

4 GPU Cluster Consensus

Distributed training and tensor-parallel inference require consensus among GPU nodes on intermediate computation results. Unlike blockchain consensus (agreeing on transaction order), GPU cluster consensus must agree on tensor values—high-dimensional floating-point data.

4.1 Tensor-Parallel Consensus

In tensor-parallel inference [4], model layers are sharded across P GPUs. Each GPU computes a partial result, and an all-reduce operation aggregates them:

$$\mathbf{h}^{(l)} = \sum_{p=1}^{P} \mathbf{h}_p^{(l)}, \tag{6}$$

where $\mathbf{h}_p^{(l)}$ is the partial activation from GPU p at layer l.

In a trusted cluster, all-reduce is a simple communication primitive. In an untrusted ACI network, a Byzantine GPU could submit incorrect partial results to corrupt the final output.

Definition 5 (Byzantine All-Reduce)**.** *A Byzantine-tolerant all-reduce protocol guarantees that the aggregated result equals the sum of honest contributions, even if up to $f < P/3$ participants submit arbitrary values.*

Algorithm 2 Byzantine All-Reduce with Commitment

Require: Partial results $\{\mathbf{h}_p\}_{p=1}^{P}$, threshold $f < P/3$
Ensure: Verified aggregate $\mathbf{h} = \sum_p \mathbf{h}_p$ (for honest p)

```
for each GPU p do
    Compute commitment: c_p = Hash(h_p)
    Broadcast c_p to all peers
end for
for each GPU p do
    After receiving all commitments, broadcast h_p
    Verify: Hash(h_p) ≟ c_p for all p
    Discard contributions with invalid commitments
end for
h = Σ_{p∈valid} h_p
```

4.2 Redundant Computation

For high-value inference (e.g., financial decisions, medical diagnoses), ACI supports redundant computation:

- The request is sent to k independent workers.
- Results are compared using the verification distance metric (§3).
- The majority result (or median for continuous outputs) is returned.
- Outlier workers are flagged for investigation.

Proposition 3 (Redundancy Security)**.** *With $k = 2f + 1$ independent workers and at most f Byzantine workers, the majority vote produces the correct output with probability 1 (assuming deterministic execution with fixed seeds).*

5 Inference Attestation

Inference attestation creates a tamper-proof on-chain record binding an inference output to its provenance: which model, which input, which GPU node, and when.

5.1 Attestation Structure

Definition 6 (Inference Attestation)**.** *An inference attestation $\mathcal{A}$ is a signed on-chain record:*

$$\mathcal{A} = (H(m), H(\mathbf{x}), H(\mathbf{y}), w, v_1, \ldots, v_t, \sigma_{threshold}, b)\,, \tag{7}$$

where $H(\cdot)$ denotes cryptographic hashes, w is the worker, $v_1, \ldots, v_t$ are the verifiers that confirmed the result, $\sigma_{threshold}$ is a threshold signature from the verification committee, and b is the block number.

5.2 Commitment Scheme

Workers commit to their outputs before verification begins, preventing them from changing outputs after seeing verification results:

1. **Commit phase**: Worker computes $c = \texttt{Hash}(m \| \mathbf{x} \| \mathbf{y} \| r)$ with random nonce r and submits c on-chain.
2. **Reveal phase**: After the verification committee is selected, the worker reveals $(m, H(\mathbf{x}), H(\mathbf{y}), r)$.
3. **Verification phase**: Verifiers re-execute and confirm or dispute the result.
4. **Finalization**: If $\geq t$ verifiers confirm, the attestation is finalized with a threshold signature.

Theorem 4 (Attestation Binding)**.** *Under the collision resistance of SHA-256, a finalized attestation $\mathcal{A}$ uniquely binds the output $\mathbf{y}$ to the model m and input $\mathbf{x}$. No polynomial-time adversary can produce a different output $\mathbf{y}' \neq \mathbf{y}$ with a valid attestation for the same $(m, \mathbf{x})$.*

5.3 Model Version Pinning

Attestations include a hash of the model weights, ensuring that results are reproducible:

$$H(m) = \texttt{SHA256}(\theta_1 \| \theta_2 \| \cdots \| \theta_L), \tag{8}$$

where θ_l are the serialized parameters of layer l. This prevents a worker from using a cheaper (smaller, distilled) model while claiming to run the requested model.

6 Distributed Training Consensus

Distributed training on ACI requires consensus on gradient aggregation across untrusted nodes.

6.1 Byzantine Gradient Aggregation

Standard distributed training uses AllReduce to average gradients:

$$\bar{\mathbf{g}} = \frac{1}{P} \sum_{p=1}^{P} \mathbf{g}_p. \tag{9}$$

A Byzantine node can submit poisoned gradients to corrupt the model. We use coordinate-wise trimmed mean [5]:

$$\bar{g}_j^{\text{trim}} = \frac{1}{P - 2f} \sum_{p=f+1}^{P-f} g_{(p),j}, \tag{10}$$

where $g_{(1),j} \leq \cdots \leq g_{(P),j}$ are the sorted gradient values for coordinate j, and f is the number of trimmed values from each end.

Theorem 5 (Trimmed Mean Convergence)**.** *Under standard assumptions (L-smooth, μ-strongly convex loss), distributed SGD with coordinate-wise trimmed mean converges at rate:*

$$\mathbb{E}[F(\mathbf{w}_T) - F(\mathbf{w}^*)] \leq \mathcal{O}\left(\frac{L}{\mu T} + \frac{f^2 \sigma^2}{\mu P^2} \right), \tag{11}$$

where σ^2 is the gradient variance and $f < P/3$ is the number of Byzantine workers.

6.2 Gradient Commitment

To prevent adaptive attacks (where a Byzantine node chooses its gradient after seeing other gradients), we use a commit-reveal scheme analogous to inference attestation:

1. Each worker commits $c_p = \texttt{Hash}(\mathbf{g}_p \| r_p)$.
2. After all commitments are received, workers reveal their gradients.
3. Gradients inconsistent with commitments are discarded.
4. Trimmed mean aggregation proceeds on valid gradients.

7 Integration with Lux Consensus

Hanzo ACI attestations are submitted to the Lux blockchain [3] for finalization. The integration leverages Lux's multi-consensus architecture:

- **Quasar consensus**: Lightweight probabilistic consensus for attestation finality, providing sub-second finalization.
- **Subnet**: ACI operates as a Lux subnet with GPU-staked validators, enabling custom consensus rules for AI workloads.
- **Cross-chain**: Attestations on the ACI subnet can be verified on the Lux primary network via cross-chain messages, enabling DeFi applications to conditionally execute based on AI inference results.

8 Production Results

Metric	Value
Active GPU nodes (workers)	240
Active verifier nodes	48
Monthly inference requests	47M
PoAI sampling rate	5%
Verification overhead	6.2% of total compute
Detected cheating attempts	23 (90-day period)
Detection success rate	99.97%
Mean attestation finality	1.4s
On-chain attestation cost	0.002 HANZO

Table 1: Production consensus metrics for Hanzo ACI (January–March 2026).

The 23 detected cheating attempts involved operators returning cached results for novel inputs (17 cases) and operators using quantized models while claiming full-precision execution (6 cases). All were detected within one verification epoch (10 minutes) and resulted in 25% stake slashing.

9 Related Work

Bittensor [6] uses a peer-ranking system (Yuma consensus) where validators score miners on output quality. Unlike PoAI, Bittensor ranking is subjective and susceptible to collusion between validators and miners.

Ritual [7] uses optimistic verification with dispute resolution. PoAI provides stronger guarantees through probabilistic sampling with economic security bounds.

Gensyn [8] focuses on training verification using probabilistic proof of learning. Our system addresses both training and inference verification.

Lux Quasar consensus [3] provides the finality layer for ACI attestations, enabling sub-second confirmation of verified inference results.

10 Conclusion

We have presented three consensus mechanisms for decentralized AI on Hanzo ACI: Proof of AI Work for probabilistic inference verification with provable economic security, GPU Cluster Consensus for Byzantine-tolerant distributed computation, and Inference Attestation for on-chain provenance binding. Production deployment across 240 GPU nodes demonstrates 99.97% cheating detection with only 6.2% verification overhead, making honest operation the only rational strategy for profit-maximizing GPU operators.

References

[1] Hanzo AI Research. Hanzo ACI: AI Chain Infrastructure for decentralized compute markets. *Hanzo AI Research*, December 2024.

[2] Hanzo AI Research. Hanzo Tokenomics: Token economics for AI infrastructure at scale. *Hanzo AI Research*, April 2026.

[3] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[4] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*, 2019.

[5] D. Yin, Y. Chen, R. Kannan, and P. Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proc. ICML*, 2018.

[6] Opentensor Foundation. Bittensor: A peer-to-peer intelligence market. *Bittensor White Paper*, 2023.

[7] Ritual. Ritual: Decentralized AI inference. *Ritual Documentation*, 2024.

[8] Gensyn. Gensyn: A protocol for deep learning computation. *Gensyn White Paper*, 2023.